

Задачи за упражнение на група 3, седмица 5

21 март 2026 г.

Задача 1 - MergeSort

Вход: Масив $A[1..n]$ от цели числа.

Изход: Масив $A[1..n]$, сортиран във възходящ ред.

Реализирайте MERGESORT като използвате MERGE от предишните задачи.

Докажете (чрез рекурентна връзка), че сложността по **време** е $O(n \log n)$ и сложността по **памят** е $O(n)$.

Рекурентна връзка:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n), \quad T(1) = O(1)$$

Задача 2 - Heapify

Дефиниция (Двоична пирамида / Мах-Хеап). Масив $A[1..n]$ е *max-heap*, ако за всеки индекс i такъв, че $2i \leq n$, е изпълнено:

$$A[i] \geq A[2i] \quad \text{и} \quad A[i] \geq A[2i + 1] \quad (\text{ако } 2i + 1 \leq n)$$

С други думи, всеки елемент е **по-голям или равен** на децата си. Масивът $A[1..n]$ представя пълно двоично дърво, в което:

- Коренът е $A[1]$.
- Децата на $A[i]$ са $A[2i]$ (ляво) и $A[2i + 1]$ (дясно).
- Родителят на $A[i]$ е $A[\lfloor i/2 \rfloor]$.

Дефиниция (Почти-куп). Масивът $A[1..n]$ е *почти-куп с нарушение в корен i* , ако:

- Лявото поддърво с корен $A[2i]$ е валиден max-heap.
- Дясното поддърво с корен $A[2i + 1]$ е валиден max-heap.
- Единствено $A[i]$ може да наруши heap-свойството спрямо децата си.

Процедура HEAPIFY(A, i, n):

Вход: Масив $A[1..n]$ — почти-куп с нарушение в корен i , и индекс i ($1 \leq i \leq n$).

Изход: Масивът A е модифициран така, че поддървото с корен i е валиден max-heap.

Ефект върху останалите елементи: Елементите извън поддървото с корен i не се променят.

Формално, след приключване на HEAPIFY(A, i, n) трябва да е изпълнено:

$$\forall j \in \{j' \mid j' \text{ е в поддървото с корен } i\} : A[\lfloor j/2 \rfloor] \geq A[j]$$

Анализ: Докажете, че HEAPIFY работи за $O(\log n)$ в най-лошия случай.

Подсказка: Нека $m = \operatorname{argmax}(A[i], A[2i], A[2i + 1])$. Ако $m \neq i$, разменете $A[i]$ с $A[m]$ и рекурсирайте върху поддървото с корен m . Броят на рекурсивните извиквания е ограничен от **височината** на дървото:

$$h = \lfloor \log_2 n \rfloor \implies T(n) = O(\log n)$$

Задача 3 - HeapSort

Вход: Масив $A[1..n]$ от цели числа.

Изход: Масив $A[1..n]$, сортиран във възходящ ред.

Реализирайте HEAPSORT в две фази, като използвате HEAPIFY от Задача 2:

1. **Фаза „Build-Heap“:** За $i = \lfloor n/2 \rfloor$ надолу до 1 извикайте HEAPIFY(A, i, n).
2. **Фаза „Extract“:** За $i = n$ надолу до 2: разменете $A[1]$ с $A[i]$, намалете размера с 1, извикайте HEAPIFY($A, 1, i-1$).

Докажете, че:

$$\text{Build-Heap} = O(n), \quad \text{HeapSort} = O(n \log n), \quad \text{памет} = O(1)$$

Задача 4 — MultitapeSort

Дадени са k сортирани масиви $T_1, T_2, \dots, T_k$ с дължини $n_1, n_2, \dots, n_k$, $\sum n_i = n$.

Задача А — итеративно: Слейте всички масиви в един сортиран масив чрез последователни извиквания на MERGE от двойка по двойка. Анализирайте времевата сложност.

Задача Б — с min-heap: Поддържайте min-heap с размер k , съдържащ текущият елемент на всяка лента. На всяка стъпка:

1. Извадете минимума от heap-а и го запишете в C .
2. Добавете следващия елемент от лентата, от която е дошъл минимумът.

Покажете, че тази стратегия работи за $O(n \log k)$.

Сравнение: Защо $O(n \log k) \leq O(n \log n)$?

Задача 5 - Maximum Sum Subarray of Size K

Вход: Масив $A[1..n]$ от цели числа и естествено число k ($1 \leq k \leq n$).

Изход: Максималната сума на *непрекъснат* подмасив с точно k елемента:

$$\max_{1 \leq i \leq n-k+1} \sum_{j=i}^{i+k-1} A[j]$$

Решете задачата за $O(n)$ с помощта на техниката „плъзгащ прозорец“.

Подсказка: Пресметнете сумата на първия прозорец $[1..k]$, след което при всяко преместване прибавете $A[i+k]$ и извадете $A[i]$.

Задача 6 - First Bad Version

Контекст. Разработвате продукт и имате n версии $1, 2, \dots, n$. В някой момент е въведена грешка, която се е *разпространила* във всички следващи версии. Разполагате с API:

$$\text{ISBADVERSION}(k) \rightarrow \{ \text{true}, \text{false} \}$$

която връща *true* ако версия k е дефектна, и *false* в противен случай.

Формално е изпълнено:

$$\exists! p \in \{1..n\} : \text{ISBADVERSION}(k) = \begin{cases} \text{false}, & k < p \\ \text{true}, & k \geq p \end{cases}$$

Вход: Естествено число n — броят версии.

Изход: Индексът p на **първата** дефектна версия.

Решете задачата като използвате **възможно най-малко** извиквания на ISBADVERSION.

Докажете, че вашето решение извиква API **най-много** $O(\log n)$ пъти.

Забележка: Не може да инспектирате кода на ISBADVERSION - тя е **черна кутия**. Единственият начин да получите информация е чрез извикване.

Подсказка: Свойството на масива $[false, \dots, false, true, \dots, true]$ е монотонно — използвайте това, за да приложите двоично търсене върху отговорите на API-то.

Контакти

При въпроси, свързани с тези или други задачи, може да се свържете с мен тук:

Instagram: [instagram.com/stanislav_.igov](https://www.instagram.com/stanislav_.igov)

Facebook: [facebook.com/stanislav.igov](https://www.facebook.com/stanislav.igov)